ADVANCED PROGRAMMING LANGUAGES

Spring 2026, CM 0632. MSc in Computer Science and Information Technology, Ca' Foscari University of Venice

Overloading and Type Classes

In the past two lectures we have studied various forms of type parametrization, both for datatypes and for the functions that operate on them. Type parametrization allows a single piece of code to be assigned a *generic* type because the code behaves *uniformly* on every instantiation of that type. The same algorithm runs irrespective of the specific types at which the type variables are instantiated.

The paradigmatic example is

```
length :: [a] -> Int
```

which acts the same way on lists of any element type. From this polymorphic type we may derive instance types such as

```
length :: [Int]    -> Int
length :: [String] -> Int
length :: [[Bool]] -> Int
...
```

Overloading, by contrast, occurs when a single function symbol is defined on several different types but acts *differently* on each of them. The classical example is the `(+)` operator, which has (at least) two distinct implementations associated with it:

```
(+) :: Int    -> Int    -> Int
(+) :: Double -> Double -> Double
```

There is no single implementation `(+) :: a -> a -> a` that works uniformly for an arbitrary type `a`. The reason we recycle the symbol `(+)` across `Int` and `Double` types is that both operations compute numeric addition — but the two algorithms are genuinely different: integers are represented as binary numbers and floating-point numbers as exponent and mantissa, so the bit-level manipulation needed to combine the operands and produce the result is unrelated in the two cases.

This is the essential contrast with parametric polymorphism. Parametric polymorphism gives *one type signature with many instances*, supported by *the same algorithm* that runs unchanged on all of them. Overloading gives *many type signatures*, each backed by *its own algorithm*; the same symbol is reused for what are, operationally, distinct functions.

As a consequence, invoking an overloaded function requires choosing among its implementations. This process — called *overloading resolution* — relies on the types of the function arguments (and possibly on the return type, which makes the resolution algorithm more involved).

The key ingredients of overloading may then be summarized as follows:

- a single function symbol refers to more than one algorithm;
- the different algorithms have different types;
- the choice of which implementation to use at each call site is determined by the type context.

Beyond arithmetic operators, several common functions are naturally overloaded rather than universally polymorphic: *equality*, *ordering*, *serialization*. Each of these is available on many types, but its implementation depends on how the values of each type are represented and therefore varies from type to type.

Toward a unifying mechanism

The pattern we have just described — a single operator with multiple, representation-specific implementations — is widespread, and it raises in two complementary ways the same need: that of denoting, as a first-class entity, a family of types.

- In one direction, the family arises by aggregation: a collection of monomorphic implementations of the same operator, originally scattered across unrelated types, is recognised as the inhabitants of one family unified by a common interface schema.
- In the other direction, the family arises by restriction: a parametrically polymorphic type, whose variable ranges too freely, is narrowed down to those types on which a required operation is defined.

Aggregation and restriction meet on the same concept, that of *type family*.

We illustrate each direction through a small example. The first, square, starts from the monomorphic implementations of `(*)` on `Int`, `Double`, and other numeric types, and asks how to recognise them as instances of a single, named family. The second, `member`, starts from the fully polymorphic type `a -> [a] -> Bool` and restricts `a` to the family of types on which equality is defined.

The two examples are not pathologies of overloading; they are the two paths along which the same notion of type family naturally enters the picture.

`square` : aggregating monomorphic implementations

Consider the function

```
square x = x * x
```

Each of the implementations of `(*)` available on a numeric type — `Int`, `Double`, `Integer`, and so on — is monomorphic: each has a fixed input and output type, and there are as many implementations as there are admissible types. We cannot give `square` the polymorphic type `a -> a -> a` in the parametric sense — that would assert uniform behaviour over *every* `a`, which is plainly false for `(*)`. And we do not want to write one copy of `square` per admissible type: with `n` numeric types in scope, the auxiliary function

```
squares x y z = (square x, square y, square z)
```

would alone require n^3 versions.

What the various implementations of `(*)` *do* share is the shape of their signature: for each admissible `a`, the operator has type `a -> a -> a`. This common shape is not yet expressible in our type system; if it were, it would let us declare `(*)` once, with a signature that reads "for each `a` in some family of admissible types, `(*)` has type `a -> a -> a`", and define `square` once on top of it. Concretely, we would need two new mechanisms: the ability to *denote a family* — say `Num a` — of the types on which `(*)` is defined, and the ability to *populate* it one type at a time, supplying for each member the corresponding implementation.

`member` : constraining a polymorphic type

Consider now the membership function on lists:

```
member :: a -> [a] -> Bool
member x []     = False
member x (y:ys) = x == y || member x ys
```

Here the situation is reversed. `member` is genuinely polymorphic over the element type `a`, and we would like it to remain so. The complication is that its body uses `(==)`, which — unlike a primitive

parametrically polymorphic operation — is defined only on those types that admit a meaningful equality test.

Two extreme positions fail to capture what we want. Giving `(==)` a parametrically polymorphic type `a -> a -> Bool` makes `member` type-check, but admits applications to types — function types, most prominently — for which equality is not even definable; some languages take this route and fall back at runtime to reference identity, which is almost never what the programmer means. Treating `(==)` as plain overloading, with separate implementations on each admissible type, blocks the very definition of `member`, because the body has no commitment to a specific implementation of `(==)`. What we want is a *constrained* form of polymorphism: `a` should range *not over all types*, but only over those for which `(==)` is defined. This is the very same family we identified for `square`, encountered from the opposite side — there we wanted to denote the family on which `(*)` is defined; here, the family on which `(==)` is defined.

Two paths converging on the same construct.

The two examples expose two distinct needs that the missing construct must serve. From `square`, the neet to *give a common interface to a collection of monomorphic implementations*: naming a family of types and the schema of operations available on each. From `member`, the neet to *constrain a polymorphic type variable* to range only over a designated family. The Haskell construct that fulfils both roles uniformly — and uniformly for arithmetic, equality, ordering, serialization, and every other family of operations we cared to identify at the end of the previous section — is the *type class*, which we introduce next.

Type Classes

A *type class* declaration introduces the name of a type family together with the interface (the *member functions*) that every type in the family must implement. *Instance* declarations populate the family one type at a time, supplying the per-type implementations of the member functions. The two mechanisms correspond exactly to the two roles anticipated above.

Generalizing monorphic types: arithmetics

We first show the case of arithmetics, where the relevant type class collects the operations we want available on every numeric type:

```
class Num a where
  (+) :: a -> a -> a
  (*) :: a -> a -> a
```

The instances, in turn, supply the implementations:

```
instance Num Int where
  (+) = addInt
  (*) = mulInt

instance Num Double where
  (+) = addDouble
  (*) = mulDouble
```

Here, `Num` names the interface schema `a -> a -> a` together with the family of types — `Int`, `Double`, and any others that supply an instance — for which the schema can be instantiated.

Constrained polymorphic types: equality

The standard example constrained polymorphism is the family of types that admit equality, captured by

```
class Eq a where
  (==) :: a -> a -> Bool
```

The declaration has two components:

- The header `class Eq a where` introduces a name `Eq` for the class and indicates that the type variable `a` will play the role of an arbitrary instance of the class in the rest of the definition.
- The body lists the *member functions* of the class. Here there is a single member, the infix operator `(==)`, whose signature `a -> a -> Bool` is the interface schema we wanted to name: for every `a` instance of `Eq`, equality is a binary operation on `a` returning a boolean.

The class declaration by itself does not yet populate the family — it only declares its shape. The family is populated by *instance declarations*:

```
instance Eq Int where
  (==) = primEqInt
```

where `primEqInt` is a built-in primitive implementing integer equality. Equality on characters can be defined in terms of equality on integers:

```
instance Eq Char where
  c == c' = ord c == ord c'
```

where `ord c` is the integer code of the character `c`.

Instances may also be given for parametric datatypes, in which case the instance itself may depend on further instances. Equality on lists is the canonical case:

```
instance Eq a => Eq [a] where
  []     == []     = True
  []     == (_:_)  = False
  (_:_)  == []     = False
  (x:xs) == (y:ys) = x == y && xs == ys
```

The qualification `Eq a =>` in the instance head states that the definition of equality on lists *depends on* equality on the element type: if `a` is an instance of `Eq`, then so is `[a]`. A finite collection of instance declarations may give rise to an infinite set of instances — `Eq Int` together with `Eq a => Eq [a]` already covers `[Int]`, `[[Int]]`, `[[[Int]]]`, and so on.

Deriving instances

For many user-defined datatypes, writing the instance by hand is mechanical. Consider the type

```
data Shape = Circle Double | Rectangle Double Double
```

An `Eq` instance for `Shape` is forced by the structure of the type: two values are equal if and only if they are built with the same constructor and the corresponding fields are pairwise equal. Writing this out gives

```
instance Eq Shape where
  Circle r       == Circle r'       = r == r'
  Rectangle w h  == Rectangle w' h' = w == w' && h == h'
  _              == _               = False
```

The implementation is determined entirely by the constructors and the equality of the field types. There is no design choice involved, only routine repetition.

Haskell offers a shortcut for these cases: the `deriving` clause, attached to a `data` declaration, instructs the compiler to *generate the instance automatically*. The example above is written more compactly as

```
data Shape = Circle Double | Rectangle Double Double
           deriving (Eq, Show)
```

which gives `Shape` instances for both `Eq` and `Show`, with the implementations produced by the compiler at compile time. The behaviour is exactly what one would expect: for `Eq`, equality is structural on constructors and recursive on fields; for `Show`, every value is rendered by printing the constructor name followed by its arguments, e.g. `Show`-printing `Circle 1.5` yields the string `"Circle 1.5"`.

A small number of standard classes can be derived in this way: `Eq` (structural equality), `Ord` (lexicographic ordering, with the order of constructors in the data declaration determining the ordering), `Show` and `Read` (printing to and parsing from strings), `Enum` and `Bounded` (for types whose constructors are nullary).

Derivation interacts naturally with qualified types. For a parametric datatype such as

```
data Tree a = Leaf | Node (Tree a) a (Tree a)
            deriving (Eq, Show)
```

the compiler does not generate an unqualified `Eq (Tree a)` — it cannot, because equality on `Tree a` depends on equality on `a`. What it generates is exactly the qualified instance

```
instance Eq a => Eq (Tree a) where
  ...
```

which mirrors the one we wrote by hand for lists in the previous subsection. More generally, deriving `C` on a type with field types `t1, ..., tn` generates an instance with context `(C t1, ..., C tn) =>` — a derivable class is only derivable on types whose components are themselves in the class, and the context propagates this requirement.

A few caveats are worth keeping in mind:

- Derivation is restricted to a closed set of standard classes. User-defined classes cannot, in general, be derived with the `deriving` mechanism alone; GHC offers extensions for this (e.g. `DeriveAnyClass`, `GeneralizedNewtypeDeriving`) which we shall not pursue.
- The generated instance is the *default* implementation — the one dictated by the structure of the type. For classes where the structural implementation is not what one wants (e.g. an `Eq` instance that ignores some field, or an `Ord` that follows a custom criterion), the manual `instance` declaration is still the right tool.

The deriving mechanism is, in this sense, a convenience layered on top of the instance system, not a separate construct: a `deriving` clause is, conceptually, a request to the compiler to write a specific instance declaration on the programmer's behalf.

Qualified Types

Once a class has been declared, every member function acquires a *qualified type*: a polymorphic type whose type variable is restricted to range over the instances of the class. The operator `(==)`, declared in the body of `class Eq a`, has global type

```
(==) :: Eq a => a -> a -> Bool
```

The notation `Eq a =>` is the *class context*: a predicate on the type variable `a`, separated from the body of the type by the symbol `=>`, which reads "such that". The whole signature is read: "for every type `a` such that `a` is an instance of `Eq`, `(==)` has type `a -> a -> Bool`". The arrow `=>` is what carries the duality identified in §1 into the syntax of types: to its left sits the family (`Eq a`), to its right sits the schema (`a -> a -> Bool`).

The same applies to the operators of `Num`:

```
(+) :: Num a => a -> a -> a
(*) :: Num a => a -> a -> a
```

Qualified types propagate through every function defined in terms of class members. The class context is *inherited*: a function that uses `(==)` cannot have a less restrictive context than `(==)` itself. So the membership function, which we could not write before, now type-checks:

```
member :: Eq a => a -> [a] -> Bool
member x []     = False
member x (y:ys) = x == y || member x ys
```

The qualification `Eq a =>` on `member` is the trace, in the signature, of the fact that the body uses an operator from `Eq`. Similarly, `square` becomes a single definition over every numeric type:

```
square :: Num a => a -> a
square x = x * x
```

A single definition — not n^3 — covers every type for which a `Num` instance is provided.

The pattern generalizes. Any function in which a class operator appears at some position acquires a class context in its signature; the context propagates outward, accumulating constraints as more class operators are used. A function that uses both `(==)` and `(+)` ends up with a context `(Eq a, Num a) =>`, signalling that its type variable must range over types that are instances of *both* classes. The system of class contexts is the *bookkeeping* that the qualified types perform on behalf of the programmer.

Implementation

Type classes and qualified types are implemented by a *compile-time translation* that takes a program containing class and instance declarations and produces an equivalent program in which neither appears. The translation has three parts: every class declaration becomes an ordinary algebraic datatype, every instance declaration becomes an ordinary value of that datatype, and every function with a qualified type becomes a function with a regular polymorphic type that takes the relevant *dictionary* as an additional parameter.

Given the class declaration of `Num`, the translation introduces a *dictionary type* — a record whose fields correspond to the member functions:

```
data NumD a = NumDict { add :: a -> a -> a,
                        mul :: a -> a -> a }
```

Each instance is then translated into a value of the dictionary type, populating its fields with the concrete implementations:

```
numDInt :: NumD Int
numDInt = NumDict { add = addInt,
                    mul = mulInt }

numDDouble :: NumD Double
numDDouble = NumDict { add = addDouble,
                       mul = mulDouble }
```

Every function whose type carries a class context is rewritten to take the corresponding dictionary as an additional argument, and to access the overloaded operators through the fields of that dictionary. The translated form of `square` is

```
square' :: NumD a -> a -> a
square' dict x = mul dict x x
```

Finally, each call site is rewritten to pass in the appropriate dictionary. Where the source program had `square 3` or `square 3.14`, the translated program has

```
square' numDInt    3       -- replaces square 3
square' numDDouble 3.14    -- replaces square 3.14
```

The choice of which dictionary to insert is made by the compiler at the call site, on the basis of the types of the arguments — this is the overloading resolution step, performed once at compile time, with no runtime cost.

For this choice to be well-defined, Haskell guarantees at the language level that there is at most one instance per pair (class, type): two instance Num Int declarations in the same program are a compile-time error. We shall see in the next lecture that Scala makes a different choice on this point.

This translation clarifies what type classes *are* and what they *are not*. They are not a new computational mechanism: anything written with type classes could, in principle, be written by hand using ordinary records and explicit dictionary passing. What type classes provide is the discipline and the automation — the compiler tracks which dictionaries are needed and threads them through the program, so that the programmer can write the natural, unencumbered form `square x = x * x` and still get the right behaviour at every type.

Type classes vs bounded polymorphism

Stepping back from the implementation, type classes can be placed in a broader picture that connects them to mechanisms we have already encountered.

In the previous lectures, we saw a different way to constrain a polymorphic type variable: *type bounds*. A bound such as `[T <: U]` restricts `T` to range only over the subtypes of `U`, so that the body of the function can rely on every operation that `U` makes available. Bounds and class contexts share the same conceptual shape — both qualify a type variable with a predicate that selects a sub-family of admissible types — but they answer the predicate by different means: a bound `[T <: U]` answers "`T` admits the operations of `U`" by appeal to *subtyping*; a context `C a =>` answers "`a` admits the operations of class `C`" by appeal to the existence of a *dictionary*.

This places type classes in their proper context. They are one form of *constrained polymorphism* among several.

- In Haskell they are the only form: the language has no subtyping, so the entire job of qualifying type variables falls on the class system.
- In Scala both mechanisms coexist — type bounds inherited from the object-oriented heritage of the language, and type classes (realized via `given` and `using`) imported from the Haskell tradition. The two are not interchangeable, and each has its sweet spot. We will return to the comparison in the next lecture, when we examine how type classes are encoded in Scala and how they relate to subtype bounds.

References and Credits

The material of this lecture is based on various sources:

- *How to make ad-hoc polymorphism less ad hoc*, a paper by Philip Wadler and Stephen Blott presented at POPL 1989, The ACM-SIGACT Symposium on Principles of Programming Languages.
- *Qualified Types: Theory and Practice*. Mark Philip Jones's PhD Dissertation. Oxford University, 1992.
- *COMP 150PLD — Programming Language Design*. Kathleen Fisher's lecture notes, Tufts University.